

Signing Github Commits: How and Why

Maggie Ring

Table of Contents

Why it's important to sign commits	Page 3
Generating a GPG keypair	Page 3
Sharing your key with GitHub.....	Page 4
Signing your commits	Page 5
Requiring signed commits for a repo	Page 5
Additional Resources	Page 6

List of Figures

Figure 1: Example of how commits can be falsified	Page 3
Figure 2: Adding a new GPG to your GitHub account	Page 5
Figure 3: Requiring contributors to sign commits	Page 6

Why it's important to sign commits

In open-source software development, maintaining a secure and trustworthy codebase is crucial. Currently, GitHub is the most popular platform for open-source software development, as well as the most targeted by threat actors. One of the most important ways to maintain safe code on GitHub is signing commits. This article will discuss how and why to sign GitHub commits.

Signing a GitHub commit is a way of verifying two things: The name on the commit is really the account that committed the code, and the code has not been tampered with. This involves using asymmetric encryption to create a private key and a public key. The private key is kept secret and used to create a digital signature that uniquely identifies the author of a commit. The public key is shared openly and used to verify the digital signature.

Signing GitHub commits is important for many reasons. Open-source software attacks are becoming increasingly common and increasingly more sophisticated, making it more important than ever to verify that commits are coming from trusted users. Just looking at the name of the account that submitted a piece of code isn't enough though. When committing to GitHub, attackers are able to falsify their commits to make it look like the code is coming from a reputable account, even linking to the trusted account when clicked on.

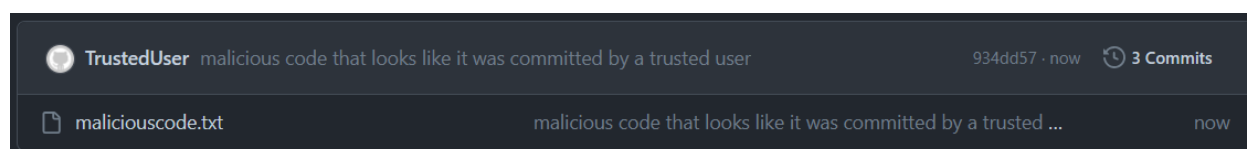


Figure 1: example of how commits can be falsified to look like they are coming from another account

Signing commits prevents this by ensuring the author of a commit is really the person who wrote the code.

Generating a GPG keypair

(This guide will only cover installation on a Linux OS.)

Besides Git, the only other prerequisite needed is GPG. It's recommended to use version 2.2 or higher. Most Linux distributions come with GPG pre-installed. If it's not, you can install the gnupg package.

For Debian and Ubuntu systems:

```
sudo apt-get install gnupg
```

For CentOS and similar OSs:

```
sudo yum install gnupg
```

Next, generate a keypair:

```
gpg --full-gen-key
```

There will be prompts for key type, size, and length of time that it is valid. You can specify these or press enter to use default values. After verifying these, enter your GitHub account information. Finally, you are prompted to give a secure passphrase that will be used to generate your keypair.

Now you need to copy your public key to share with GitHub. First, list your GPG keypairs:

```
gpg --list-secret-keys --keyid-format=long
```

Copy the long form of the GPG key ID that you want to use. Then, use this ID to print out the public key:

```
gpg --armor --export [GPG key ID]
```

Finally, copy your public key. It should start with:

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
```

And end with:

```
-----END PGP PUBLIC KEY BLOCK-----
```

Sharing your key with GitHub

Now you can add the public GPG key you generated to your GitHub account.

- Click on your **profile**, then **settings**
- In the **Access** section of the sidebar, click **SSH and GPG keys**
- Click **New GPG key**
- Add a name for your key in the **title** field
- Paste the public GPG key in the **key** field
- Click **Add GPG key**

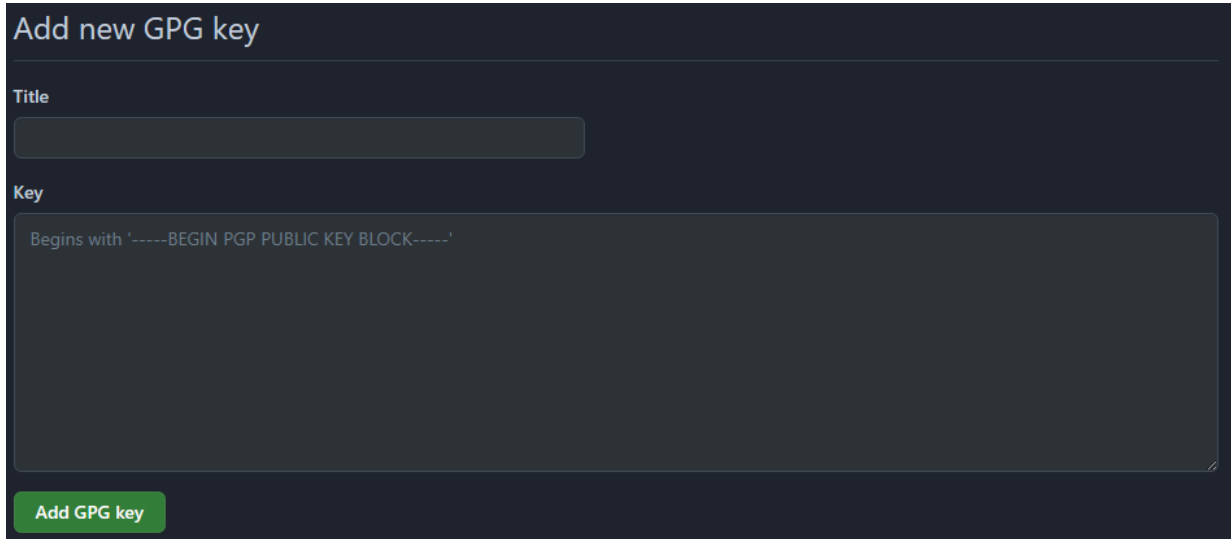


Figure 2: Adding a new GPG to your GitHub account

Signing your commits

Now that GitHub can link you to your commits, all you have to do is use the `-S` flag when committing:

```
git commit -S -m "YOUR_COMMIT_MESSAGE"
```

After pushing a signed commit, you should see a green checkmark and the word **verified** by your commit.

Requiring signed commits for a repo

Now that you are signing your own commits, you might want to require anyone else who contributes to a repository you own to sign their commits too. This can be done in a couple of steps:

- Navigate to the main page of your repository
- Click **settings**
- Under **General**, Select **Require contributors to sign off on web-based commits**

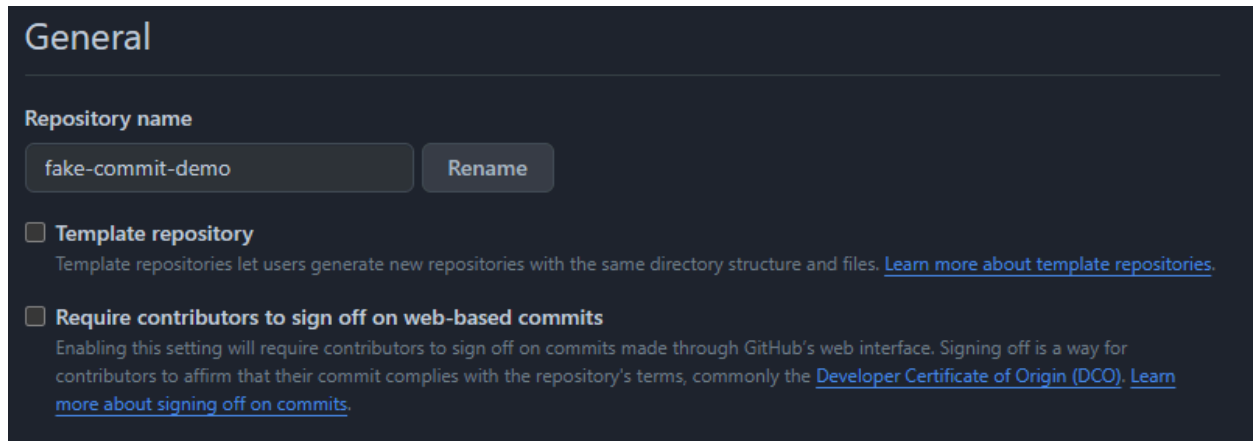


Figure 3: Requiring contributors to sign commits

Additional resources

For additional resources like troubleshooting, signing a key on a non Linux OS, or using other types of keys, see the [GitHub Docs](#).